Python Engineering - Reference Guide

Class 10 - Advanced Functions: arguments and functions as objects

A simple but extended guide to function argument styles and to the idea that Python functions can also be passed as data.

1. Parameter vs argument

A **parameter** is the name written in the function definition. An **argument** is the actual value passed when calling it.

```
def energy(mass, velocity):   # parameters
    ...

energy(10, 50)                # arguments
```

2. Positional arguments

What they do: assign values according to call order.

What they receive: comma-separated values.

What they produce: each value is assigned to the parameter at the same position.

```
def initial_energy(h0, v0, g):
    ...

initial_energy(0.0, 50.0, 9.81)
```

3. Keyword arguments

What they do: assign values by parameter name rather than position.

What they receive: name=value pairs.

What they produce: values are matched to parameters by name.

```
initial_energy(v0=50.0, g=9.81, h0=0.0)
```

4. Default arguments

What they do: provide a fallback value when the caller omits that argument.

How they are defined: parameter=value.

What they produce: the function can be called without explicitly providing that value.

```
def initial_energy(h0, v0, g=9.81):
    ...

initial_energy(0.0, 50.0)
initial_energy(0.0, 50.0, g=1.62)
```

5. *args in a DEFINITION

What it does: collects a variable number of positional arguments.

What it receives: zero, one, or many positional values.

What it becomes inside the function: a **tuple**.

```
def average_measurements(*values):
    print(type(values))
    print(values)

average_measurements(10, 20, 30)
# tuple: (10, 20, 30)
```

6. **kwargs in a DEFINITION

What it does: collects a variable number of named arguments.

What it receives: name=value pairs.

What it becomes inside the function: a **dictionary**.

```
def summary(**kwargs):
    print(kwargs)

summary(h=120, v=-10)
# {'h': 120, 'v': -10}
```

7. * when CALLING: unpacking

What it does: unpacks a sequence into positional arguments.

What it receives: commonly a list, tuple, or iterable sequence.

What it produces: a call equivalent to writing each value separately.

```
values = [10, 20, 30]
average_measurements(*values)

# same as:
average_measurements(10, 20, 30)
```

8. ** when CALLING: dictionary unpacking

What it does: unpacks dictionary entries into keyword arguments.

What it receives: a dictionary with string keys.

What it produces: a keyword-style call; keys must match expected parameter names.

```
config = {'h0':0.0, 'v0':50.0, 'g':9.81}
initial_energy(**config)

# same as:
initial_energy(h0=0.0, v0=50.0, g=9.81)
```

9. Definition vs call

Syntax	In def	In call
*name	collect positional args into tuple	unpack a sequence
**name	collect keyword args into dict	unpack a dictionary

10. Passing a function as an argument

In Python a function is also an object. It can be assigned, stored, or passed to another function.

```
def add(a, b):
    return a + b

def apply_operation(operation, x, y):
    return operation(x, y)

apply_operation(add, 5, 3)
# 8
```

operation receives the add function itself. Later, operation(x, y) calls whichever function was received.

11. Function object vs function call

```
add      # function object
add()    # call the function now
```

We pass add without parentheses when we want to pass the function itself. Parentheses execute it immediately.

12. Why this matters for numerical engineering

```
solve_root(function, ...)
integrate(function, ...)
solve_ode(function, ...)
```

A numerical algorithm can receive a mathematical function and evaluate it whenever required, without being hard-coded to one equation.

13. Quick summary

Concept	Input	Result / behavior
Positional	values	assigned by order
Keyword	name=value	assigned by name
Default	parameter=default	argument may be omitted
*args	many positional values	tuple inside function
**kwargs	many named values	dict inside function
*sequence	list/tuple	positional unpacking
**dictionary	dict	keyword unpacking
function as argument	function without ()	another function can call it